

```

"""
Chat Server – Professional Multi-threaded TCP Server
Supports: Text messaging, File transfer, Group rooms, Video & Voice
calls
Protocol: Newline-delimited headers with length-specified binary
payloads
"""

import socket
import threading
import time
import traceback

HOST = '0.0.0.0'
PORT = 5555

clients = {}          # username -> socket
rooms = {}            # room_name -> [user1, user2, ...]
active_calls = {}     # username -> (partner, call_type)  e.g. 'alice'
                      -> ('bob', 'video')

# ----- Buffered Receiver -----

class BufferedReader:
    """Reliably reads newline-delimited headers and binary payloads
    from a TCP stream."""

    def __init__(self, sock):
        self.sock = sock
        self.buffer = b""

    def read_line(self):
        """Read bytes until '\\n' and return the decoded line
        (without newline)."""
        while b'\n' not in self.buffer:
            data = self.sock.recv(4096)
            if not data:
                raise ConnectionError("Connection closed")
            self.buffer += data
        idx = self.buffer.index(b'\n')
        line = self.buffer[:idx].decode('utf-8', errors='replace')

```

```

        self.buffer = self.buffer[idx + 1:]
        return line

def read_bytes(self, size):
    """Read exactly *size* bytes from the stream."""
    while len(self.buffer) < size:
        remaining = size - len(self.buffer)
        data = self.sock.recv(min(4096, remaining))
        if not data:
            raise ConnectionError("Connection closed")
        self.buffer += data
    result = self.buffer[:size]
    self.buffer = self.buffer[size:]
    return result

# ----- Helper -----
def send_to(user, message_bytes):
    """Safely send raw bytes to a user's socket."""
    if user in clients:
        try:
            clients[user].sendall(message_bytes)
        except Exception:
            try:
                del clients[user]
            except KeyError:
                pass

def update_lists():
    users = ",".join(clients.keys())
    room_list = ",".join(rooms.keys())
    for c in list(clients.values()):
        try:
            c.sendall(f"LIST|{users}\n".encode())
            time.sleep(0.01)
            c.sendall(f"ROOMS|{room_list}\n".encode())
        except Exception:
            pass

# ----- Client Handler -----
def handle_client(conn, addr):

```

```

reader = BufferedReader(conn)
username = None

try:
    # First line is the username
    username = reader.read_line().strip()
    clients[username] = conn
    print(f"[+] {username} connected from {addr}")
    update_lists()

while True:
    header = reader.read_line()
    if not header:
        break

    parts = header.split('|')
    mtype = parts[0]

    # —— GROUP MESSAGE ——
    if mtype == "GROUP":
        room, msg = parts[1], parts[2]
        if room in rooms:
            if username not in rooms[room]:
                conn.sendall(f"[SYSTEM] You must join the
room first.\n".encode())
                continue
            for user in rooms[room]:
                if user != username and user in clients:
                    send_to(user, f"[{room}] {username}:
{msg}\n".encode())

    # —— ROOM MANAGEMENT ——
    elif mtype == "CREATE":
        room = parts[1]
        rooms[room] = [username]
        update_lists()

    elif mtype == "JOIN":
        room = parts[1]
        if room in rooms and username not in rooms[room]:
            rooms[room].append(username)
            update_lists()

```

```

elif mtype == "LEAVE":
    room = parts[1]
    if room in rooms and username in rooms[room]:
        rooms[room].remove(username)
    if not rooms[room]:
        del rooms[room]
    update_lists()

# —— PRIVATE MESSAGE ——
elif mtype == "PRIVATE":
    target, msg = parts[1], parts[2]
    send_to(target, f"[PRIVATE] {username}:
{msg}\n".encode())

# —— FORWARD ——
elif mtype == "FORWARD":
    target, msg = parts[1], parts[2]
    send_to(target, f"[FORWARDED] {username}:
{msg}\n".encode())

# —— FILE TRANSFER ——
elif mtype == "FILE":
    target = parts[1]
    fname = parts[2]
    size = int(parts[3])
    data = reader.read_bytes(size)
    # Forward to target with sender as source
    fwd_header = f"FILE|{username}|{fname}|{size}\n"
    send_to(target, fwd_header.encode() + data)

# —— VIDEO / VOICE DATA (only during active calls)
——

elif mtype in ("VIDEO", "VOICE"):
    target = parts[1]
    size = int(parts[3])
    data = reader.read_bytes(size)
    # Only relay if call is active between these two
users

    call_info = active_calls.get(username)
    if call_info and call_info[0] == target:
        fwd_header = f"{mtype}|{username}|x|{size}\n"

```

```

        send_to(target, fwd_header.encode() + data)

# —— CALL SIGNALING ——
elif mtype == "VIDCALL_REQ":
    target = parts[1]
    call_type = parts[2] if len(parts) > 2 else 'video'
    send_to(target,
f"VIDCALL_REQ|{username}|{call_type}\n".encode())

    elif mtype == "VIDCALL_ACC":
        target = parts[1]
        call_type = parts[2] if len(parts) > 2 else 'video'
        # Register active call for BOTH users
        active_calls[username] = (target, call_type)
        active_calls[target] = (username, call_type)
        send_to(target,
f"VIDCALL_ACC|{username}|{call_type}\n".encode())

    elif mtype == "VIDCALL_REJ":
        target = parts[1]
        call_type = parts[2] if len(parts) > 2 else 'video'
        send_to(target,
f"VIDCALL_REJ|{username}|{call_type}\n".encode())

    elif mtype == "VIDCALL_END":
        target = parts[1]
        # Remove active call entries
        active_calls.pop(username, None)
        active_calls.pop(target, None)
        send_to(target,
f"VIDCALL_END|{username}\n".encode())

except ConnectionError:
    pass
except Exception as e:
    print(f"[!] Error handling {username or 'unknown'}: {e}")
    traceback.print_exc()
finally:
    if username:
        # Clean up
        call_info = active_calls.pop(username, None)
        if call_info:

```

```

        partner = call_info[0]
        active_calls.pop(partner, None)
        send_to(partner,
f"VIDCALL_END|{username}\n".encode())
        clients.pop(username, None)
        # Remove from rooms
        for room in list(rooms.keys()):
            if username in rooms[room]:
                rooms[room].remove(username)
            if not rooms[room]:
                del rooms[room]
        conn.close()
        update_lists()
        print(f"[-] {username} disconnected")

# ----- Main -----
def start():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(50)
    print(f"[*] Chat Server running on {HOST}:{PORT}")

    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr),
daemon=True).start()

if __name__ == "__main__":
    start()

```

""""

Chat Client — Professional WhatsApp-like GUI

Supports: Text messaging, File transfer, Group rooms, Video & Voice calls

Protocol: Newline-delimited headers with length-specified binary payloads

""""

```
import socket
import threading
import tkinter as tk
from tkinter import messagebox, filedialog, simpledialog
import cv2
import pickle
import os
import time
import io
import struct
import queue
from datetime import datetime
from PIL import Image, ImageTk
try:
    import pyaudio
    HAS_PYAUDIO = True
except ImportError:
    HAS_PYAUDIO = False
```

————— Buffered Receiver —————

```
class BufferedReader:
```

"""Reliably reads newline-delimited headers and binary payloads from a TCP stream."""

```
def __init__(self, sock):
```

```
    self.sock = sock
```

```
    self.buffer = b""
```

```
def read_line(self):
```

```
    while b'\n' not in self.buffer:
```

```
        data = self.sock.recv(4096)
```

```
        if not data:
```

```
            raise ConnectionError("Connection closed")
```

```
        self.buffer += data
```

```
    idx = self.buffer.index(b'\n')
```

```
    line = self.buffer[:idx].decode('utf-8', errors='replace')
```

```
    self.buffer = self.buffer[idx + 1:]
```

```
    return line
```

```
def read_bytes(self, size):
```

```
    while len(self.buffer) < size:
```

```
        remaining = size - len(self.buffer)
```

```
        data = self.sock.recv(min(8192, remaining))
```

```
        if not data:
```

```
            raise ConnectionError("Connection closed")
```

```
        self.buffer += data
```

```
    result = self.buffer[:size]
```

```
    self.buffer = self.buffer[size:]
```

```
    return result
```

————— Color Palette (WhatsApp Dark) —————

```
class C:
```

```
BG_MAIN      = "#111b21"
BG_SIDEBAR   = "#202c33"
BG_CHAT      = "#0b141a"
BG_INPUT     = "#202c33"
BG_MSG_SENT  = "#005c4b"
BG_MSG_RECV  = "#202c33"
BG_MSG_SYS   = "#1b3a4b"
BG_HOVER     = "#2a3942"
BG_SELECTED  = "#2a3942"
ACCENT       = "#00a884"
ACCENT_HOVER = "#06cf9c"
TEXT_PRIMARY = "#e9edef"
TEXT_SECONDARY = "#8696a0"
TEXT_DARK    = "#111b21"
BORDER       = "#2a3942"
ONLINE       = "#00a884"
CALL_GREEN   = "#25d366"
CALL_RED     = "#ea0038"
CALL_BG      = "#1a2730"
WHITE        = "#ffffff"
UNREAD_BG    = "#00a884"
```

_____ Chat Client _____

```
class ChatClient:
```

```
    def __init__(self, root):
        self.root = root
        self.root.title("WhatsApp Pro")
        self.root.geometry("1200x750")
        self.root.configure(bg=C.BG_MAIN)
        self.root.minsize(900, 600)
```

```
# Try to set icon
```

```
try:
```

```
    self.root.iconname("WhatsApp Pro")
```

```
except Exception:
```

```
    pass
```

```
# Login
```

```
self.username = simplifiedialog.askstring("Login", "Enter your username:", parent=root)
```

```
if not self.username:
```

```
    root.destroy()
```

```
    return
```

```
# Network
```

```
self.SERVER_HOST = '127.0.0.1'
```

```
self.SERVER_PORT = 5555
```

```
try:
```

```
    self.client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
    self.client.connect((self.SERVER_HOST, self.SERVER_PORT))
```

```
    self.client.sendall(f"{self.username}\n".encode())
```

```
except Exception as e:
```

```
    messagebox.showerror("Connection Error", f"Cannot connect to server:\n{e}")
```

```
    root.destroy()
```

```
    return
```

```
# State
```

```
self.chat_items = {}    # chat_name -> frame widget (MUST be before UI setup)
```

```
self.chats = {}         # chat_name -> [(msg, sender, is_img), ...]
```

```
self.current_chat = None
```

```
self.unread = {}        # chat_name -> count
```

```
self.chat_widgets = {} # chat_name -> last label widget (for preview)

# Call state
self.call_active = False
self.call_partner = None
self.call_type = None # 'video' or 'voice'
self.call_window = None
self.call_start_time = None
self._stop_call = threading.Event()

# Video/Audio state
self.local_cap = None
self.local_photo = None
self.remote_photo = None
self.local_label_widget = None
self.remote_label_widget = None
self.mic_muted = False

# Media queue for received data
self.media_queue = queue.Queue()

# Build UI
self.setup_ui()

# Start background threads
threading.Thread(target=self.receive_loop, daemon=True).start()
threading.Thread(target=self.media_handler, daemon=True).start()

# PyAudio warning
if not HAS_PYAUDIO:
    self.root.after(500, lambda: messagebox.showwarning("Audio Unavailable",
```

"PyAudio is not installed. Voice/Video calls will not have audio.\n"

"Install with: pip install pyaudio"))

```
# =====  
  
#          UI SETUP  
  
# =====  
  
def setup_ui(self):  
    # Main container  
  
    self.main_frame = tk.Frame(self.root, bg=C.BG_MAIN)  
    self.main_frame.pack(fill=tk.BOTH, expand=True)  
  
  
    # — LEFT SIDEBAR —  
  
    self.sidebar = tk.Frame(self.main_frame, bg=C.BG_SIDEBAR, width=320)  
    self.sidebar.pack(side=tk.LEFT, fill=tk.Y)  
    self.sidebar.pack_propagate(False)  
  
    self._build_sidebar()  
  
  
    # — VERTICAL DIVIDER —  
  
    divider = tk.Frame(self.main_frame, bg=C.BORDER, width=1)  
    divider.pack(side=tk.LEFT, fill=tk.Y)  
  
  
    # — RIGHT CHAT PANEL —  
  
    self.chat_panel = tk.Frame(self.main_frame, bg=C.BG_CHAT)  
    self.chat_panel.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)  
  
  
    self._build_chat_panel()  
  
  
    # Show welcome screen  
  
    self._show_welcome()
```

```

# ----- SIDEBAR -----

def _build_sidebar(self):

    # Header

    hdr = tk.Frame(self.sidebar, bg=C.BG_SIDEBAR, height=60)

    hdr.pack(fill=tk.X)

    hdr.pack_propagate(False)

    tk.Label(hdr, text=f" {self.username}", font=("Segoe UI", 16, "bold"),
             bg=C.BG_SIDEBAR, fg=C.TEXT_PRIMARY).pack(side=tk.LEFT, padx=10, pady=10)

    # Online indicator

    online_dot = tk.Canvas(hdr, width=12, height=12, bg=C.BG_SIDEBAR, highlightthickness=0)

    online_dot.pack(side=tk.LEFT, padx=(0, 5))

    online_dot.create_oval(2, 2, 10, 10, fill=C.ONLINE, outline=C.ONLINE)

    # Search bar

    search_frame = tk.Frame(self.sidebar, bg=C.BG_SIDEBAR)

    search_frame.pack(fill=tk.X, padx=10, pady=(5, 10))

    self.search_var = tk.StringVar()

    self.search_var.trace_add("write", lambda *_: self._filter_chats())

    search_entry = tk.Entry(search_frame, textvariable=self.search_var,
                           font=("Segoe UI", 11), bg=C.BG_CHAT, fg=C.TEXT_PRIMARY,
                           insertbackground=C.TEXT_PRIMARY, relief=tk.FLAT,
                           highlightthickness=1, highlightcolor=C.ACCENT,
                           highlightbackground=C.BORDER)

    search_entry.pack(fill=tk.X, ipady=6)

    search_entry.insert(0, " 🔍 Search or start new chat")

    search_entry.bind("<FocusIn>", lambda e: search_entry.delete(0, tk.END) if
search_entry.get().startswith(" 🔍 ") else None)

```

```
search_entry.bind("<FocusOut>", lambda e: search_entry.insert(0, " 🔍 Search or start new chat") if not search_entry.get() else None)
```

```
# Chat list (scrollable)
```

```
list_container = tk.Frame(self.sidebar, bg=C.BG_SIDEBAR)
```

```
list_container.pack(fill=tk.BOTH, expand=True)
```

```
self.chat_list_canvas = tk.Canvas(list_container, bg=C.BG_SIDEBAR, highlightthickness=0)
```

```
scrollbar = tk.Scrollbar(list_container, orient=tk.VERTICAL,  
command=self.chat_list_canvas.yview,
```

```
bg=C.BG_SIDEBAR, troughcolor=C.BG_SIDEBAR)
```

```
self.chat_list_frame = tk.Frame(self.chat_list_canvas, bg=C.BG_SIDEBAR)
```

```
self.chat_list_frame.bind("<Configure>",
```

```
lambda e: self.chat_list_canvas.configure(scrollregion=self.chat_list_canvas.bbox("all")))
```

```
self.chat_list_canvas.create_window((0, 0), window=self.chat_list_frame, anchor="nw")
```

```
self.chat_list_canvas.configure(yscrollcommand=scrollbar.set)
```

```
self.chat_list_canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
```

```
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
```

```
# Bind mouse wheel
```

```
self.chat_list_canvas.bind_all("<MouseWheel>",
```

```
lambda e: self.chat_list_canvas.yview_scroll(-1 * (e.delta // 120), "units"))
```

```
# Store chat items (already initialized in __init__)
```

```
# ————— CHAT PANEL —————
```

```
def _build_chat_panel(self):
```

```
# Chat header
```

```
self.chat_header = tk.Frame(self.chat_panel, bg=C.BG_SIDEBAR, height=65)
```

```
self.chat_header.pack(fill=tk.X)
```

```

self.chat_header.pack_propagate(False)

self.header_name = tk.Label(self.chat_header, text="Select a chat",
                             font=("Segoe UI", 15, "bold"),
                             bg=C.BG_SIDEBAR, fg=C.TEXT_PRIMARY)
self.header_name.pack(side=tk.LEFT, padx=20, pady=15)

self.header_status = tk.Label(self.chat_header, text="",
                               font=("Segoe UI", 10),
                               bg=C.BG_SIDEBAR, fg=C.TEXT_SECONDARY)
self.header_status.pack(side=tk.LEFT, padx=(0, 10), pady=15)

# Call buttons in header
btn_frame = tk.Frame(self.chat_header, bg=C.BG_SIDEBAR)
btn_frame.pack(side=tk.RIGHT, padx=15, pady=12)

self.video_call_btn = tk.Button(btn_frame, text="📞", font=("Segoe UI", 14),
                                bg=C.BG_SIDEBAR, fg=C.TEXT_PRIMARY, relief=tk.FLAT,
                                cursor="hand2", command=self.video_call,
                                activebackground=C.BG_HOVER, activeforeground=C.ACCENT)
self.video_call_btn.pack(side=tk.LEFT, padx=5)

self.voice_call_btn = tk.Button(btn_frame, text="📞", font=("Segoe UI", 14),
                                bg=C.BG_SIDEBAR, fg=C.TEXT_PRIMARY, relief=tk.FLAT,
                                cursor="hand2", command=self.voice_call,
                                activebackground=C.BG_HOVER, activeforeground=C.ACCENT)
self.voice_call_btn.pack(side=tk.LEFT, padx=5)

# Messages area
msg_container = tk.Frame(self.chat_panel, bg=C.BG_CHAT)
msg_container.pack(fill=tk.BOTH, expand=True)

```

```

self.msg_canvas = tk.Canvas(msg_container, bg=C.BG_CHAT, highlightthickness=0)

msg_scrollbar = tk.Scrollbar(msg_container, orient=tk.VERTICAL,
command=self.msg_canvas.yview,

                                bg=C.BG_CHAT, troughcolor=C.BG_CHAT)

self.msg_frame = tk.Frame(self.msg_canvas, bg=C.BG_CHAT)

self.msg_frame.bind("<Configure>",

    lambda e: self.msg_canvas.configure(scrollregion=self.msg_canvas.bbox("all")))

self.msg_canvas.create_window((0, 0), window=self.msg_frame, anchor="nw", width=850)

self.msg_canvas.configure(yscrollcommand=msg_scrollbar.set)


self.msg_canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

msg_scrollbar.pack(side=tk.RIGHT, fill=tk.Y)


# Input area

input_area = tk.Frame(self.chat_panel, bg=C.BG_INPUT, height=65)

input_area.pack(fill=tk.X, side=tk.BOTTOM)

input_area.pack_propagate(False)


# Attachment button

attach_btn = tk.Button(input_area, text="📎", font=("Segoe UI", 16),

                        bg=C.BG_INPUT, fg=C.TEXT_SECONDARY, relief=tk.FLAT,

                        cursor="hand2", command=self.send_file,

                        activebackground=C.BG_HOVER, activeforeground=C.ACCENT)

attach_btn.pack(side=tk.LEFT, padx=(10, 5), pady=10)


# Room buttons

room_frame = tk.Frame(input_area, bg=C.BG_INPUT)

room_frame.pack(side=tk.LEFT, padx=5, pady=10)

```

```

room_btn = tk.Button(room_frame, text="🏠 Room", font=("Segoe UI", 9),
                      bg=C.BG_INPUT, fg=C.TEXT_SECONDARY, relief=tk.FLAT,
                      cursor="hand2", command=self._room_menu,
                      activebackground=C.BG_HOVER, activeforeground=C.ACCENT)
room_btn.pack(side=tk.LEFT, padx=2)

# Text entry
entry_frame = tk.Frame(input_area, bg=C.BG_CHAT)
entry_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=5, pady=10)

self.entry = tk.Entry(entry_frame, font=("Segoe UI", 12), bg=C.BG_CHAT,
                      fg=C.TEXT_PRIMARY, insertbackground=C.TEXT_PRIMARY,
                      relief=tk.FLAT, highlightthickness=1,
                      highlightcolor=C.ACCENT, highlightbackground=C.BORDER)
self.entry.pack(fill=tk.BOTH, expand=True, ipady=5)
self.entry.bind('<Return>', lambda e: self.send_message())

# Send button
self.send_btn = tk.Button(input_area, text="➤", font=("Segoe UI", 18),
                          bg=C.ACCENT, fg=C.WHITE, relief=tk.FLAT,
                          cursor="hand2", command=self.send_message,
                          activebackground=C.ACCENT_HOVER, width=3)
self.send_btn.pack(side=tk.RIGHT, padx=(5, 12), pady=10)

# ————— WELCOME SCREEN —————
def _show_welcome(self):
    for w in self.msg_frame.winfo_children():
        w.destroy()
    welcome = tk.Label(self.msg_frame,
                      text="WhatsApp Pro\n\nSelect a chat to start messaging.\n\n"
                          "Features: Text • Files • Video Call • Voice Call • Group Rooms",

```

[illegible]

```
info_frame = tk.Frame(item_frame, bg=C.BG_SIDEBAR)
info_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 10))
```

```
name_label = tk.Label(info_frame, text=name, font=("Segoe UI", 13),
                      bg=C.BG_SIDEBAR, fg=C.TEXT_PRIMARY, anchor="w")
name_label.pack(fill=tk.X, pady=(8, 0))
```

```
preview_label = tk.Label(info_frame, text="", font=("Segoe UI", 10),
                        bg=C.BG_SIDEBAR, fg=C.TEXT_SECONDARY, anchor="w")
preview_label.pack(fill=tk.X, pady=(0, 5))
```

Unread badge

```
badge_label = tk.Label(info_frame, text="", font=("Segoe UI", 9, "bold"),
                      bg=C.UNREAD_BG, fg=C.WHITE, width=3)
```

Don't pack yet — only show when unread > 0

Hover effect

```
def on_enter(e):
```

```
    item_frame.configure(bg=C.BG_HOVER)
    avatar_frame.configure(bg=C.BG_HOVER)
    info_frame.configure(bg=C.BG_HOVER)
    name_label.configure(bg=C.BG_HOVER)
    preview_label.configure(bg=C.BG_HOVER)
```

```
def on_leave(e):
```

```
    bg = C.BG_SELECTED if self.current_chat == name else C.BG_SIDEBAR
    item_frame.configure(bg=bg)
    avatar_frame.configure(bg=bg)
    info_frame.configure(bg=bg)
    name_label.configure(bg=bg)
    preview_label.configure(bg=bg)
```

```

item_frame.bind("<Enter>", on_enter)
item_frame.bind("<Leave>", on_leave)
for child in item_frame.winfo_children():
    child.bind("<Enter>", on_enter)
    child.bind("<Leave>", on_leave)
    for grandchild in child.winfo_children():
        grandchild.bind("<Enter>", on_enter)
        grandchild.bind("<Leave>", on_leave)

```

Click handler

```

def on_click(e):
    self.open_chat(name)
item_frame.bind("<Button-1>", on_click)
for child in item_frame.winfo_children():
    child.bind("<Button-1>", on_click)
    for grandchild in child.winfo_children():
        grandchild.bind("<Button-1>", on_click)

```

```

self.chat_items[name] = {
    'frame': item_frame,
    'name_label': name_label,
    'preview_label': preview_label,
    'badge_label': badge_label,
    'is_room': is_room
}

```

```

def _update_chat_preview(self, name, text):
    """Update the last message preview in sidebar."""
    if name in self.chat_items:
        preview = text[:40] + "..." if len(text) > 40 else text

```

```
self.chat_items[name]['preview_label'].config(text=preview)
```

```
def _update_unread_badge(self, name):
```

```
    """Show/hide unread badge."""
```

```
    if name not in self.chat_items:
```

```
        return
```

```
    count = self.unread.get(name, 0)
```

```
    badge = self.chat_items[name]['badge_label']
```

```
    if count > 0:
```

```
        badge.config(text=str(count))
```

```
        badge.pack(side=tk.RIGHT, padx=10, pady=(0, 5))
```

```
    else:
```

```
        badge.pack_forget()
```

```
def _highlight_active_chat(self):
```

```
    """Highlight the currently selected chat in sidebar."""
```

```
    for name, item in self.chat_items.items():
```

```
        bg = C.BG_SELECTED if name == self.current_chat else C.BG_SIDEBAR
```

```
        item['frame'].configure(bg=bg)
```

```
        for child in item['frame'].winfo_children():
```

```
            if isinstance(child, tk.Frame):
```

```
                child.configure(bg=bg)
```

```
                for gc in child.winfo_children():
```

```
                    if isinstance(gc, (tk.Label, tk.Frame)):
```

```
                        gc.configure(bg=bg)
```

```
def _filter_chats(self):
```

```
    """Filter chat list by search text."""
```

```
    query = self.search_var.get().lower()
```

```
    for name, item in self.chat_items.items():
```

```
        if query in name.lower():
```

```

        item['frame'].pack(fill=tk.X, pady=0)
    else:
        item['frame'].pack_forget()

# =====
#          CHAT OPEN
# =====

def open_chat(self, name):
    self.current_chat = name
    self._highlight_active_chat()

    # Clear unread
    if name in self.unread:
        self.unread[name] = 0
        self._update_unread_badge(name)

    # Update header
    display_name = name
    self.header_name.config(text=display_name)
    status_text = "online" if not name.startswith("Room:") else "group"
    self.header_status.config(text=status_text)

    # Clear messages
    for w in self.msg_frame.winfo_children():
        w.destroy()

    # Load chat history
    if name in self.chats:
        for msg, sender, is_img in self.chats[name]:
            self._display_message(msg, sender, is_img)

```

```

# Scroll to bottom

self.msg_frame.update_idletasks()

self.msg_canvas.yview_moveto(1.0)


# =====
#             DISPLAY MESSAGES
# =====

def _display_message(self, msg, sender, is_img=False):
    # Row container

    row = tk.Frame(self.msg_frame, bg=C.BG_CHAT)
    row.pack(fill=tk.X, pady=2, padx=15)


    if sender == "me":
        align = "e"

        bg = C.BG_MSG_SENT
        fg = C.TEXT_PRIMARY
        anchor = "e"

        side = tk.RIGHT
    elif sender == "system":
        align = "center"

        bg = C.BG_MSG_SYS
        fg = C.TEXT_SECONDARY
        anchor = "center"

        side = None
    else:
        align = "w"

        bg = C.BG_MSG_RECV
        fg = C.TEXT_PRIMARY
        anchor = "w"

        side = tk.LEFT

```

```

if is_img:

    bubble = tk.Frame(row, bg=bg, padx=8, pady=6)

    if side:

        bubble.pack(side=side, anchor=anchor)

    else:

        bubble.pack(anchor=anchor)

    label = tk.Label(bubble, image=msg, bg=bg)

    label.image = msg

    label.pack()

    dl_btn = tk.Button(bubble, text=" Save", font=("Segoe UI", 8),
                        bg=bg, fg=C.ACCENT, relief=tk.FLAT, cursor="hand2",
                        command=lambda m=msg: self._save_chat_image(m))

    dl_btn.pack(pady=(4, 0))

elif isinstance(msg, dict) and msg.get('file'):

    bubble = tk.Frame(row, bg=bg, padx=12, pady=8)

    if side:

        bubble.pack(side=side, anchor=anchor)

    else:

        bubble.pack(anchor=anchor)

    tk.Label(bubble, text=f" {msg['text']}", font=("Segoe UI", 11),
              bg=bg, fg=fg, wraplength=350, justify=tk.LEFT).pack(side=tk.LEFT)

    tk.Button(bubble, text="", font=("Segoe UI", 12),
              bg=bg, fg=C.ACCENT, relief=tk.FLAT, cursor="hand2",
              command=lambda: self._save_file(msg['name'], msg['data'])).pack(side=tk.LEFT, padx=5)

else:

    # System messages centered

```

```

if sender == "system":
    bubble = tk.Frame(row, bg=bg, padx=12, pady=6)
    bubble.pack(anchor=anchor)
    tk.Label(bubble, text=msg, font=("Segoe UI", 10, "italic"),
             bg=bg, fg=fg, wraplength=400).pack()
else:
    bubble = tk.Frame(row, bg=bg, padx=12, pady=8)
    if side:
        bubble.pack(side=side, anchor=anchor)
    else:
        bubble.pack(anchor=anchor)
    tk.Label(bubble, text=msg, font=("Segoe UI", 11),
             bg=bg, fg=fg, wraplength=400, justify=tk.LEFT).pack()

# Auto-scroll
self.msg_frame.update_idletasks()
self.msg_canvas.yview_moveto(1.0)

def _add_message(self, chat, msg, sender, is_img=False):
    """Add a message to chat history and display if current chat."""
    if chat not in self.chats:
        self.chats[chat] = []
    self.chats[chat].append((msg, sender, is_img))

# Update preview
preview_text = msg if isinstance(msg, str) else (msg.get('text', 'Image') if isinstance(msg, dict)
else 'Image')
self._update_chat_preview(chat, preview_text)

if chat == self.current_chat:
    self._display_message(msg, sender, is_img)

```

```

else:

    self.unread[chat] = self.unread.get(chat, 0) + 1

    self._update_unread_badge(chat)

# =====

#          SEND / RECEIVE

# =====

def send_message(self):

    msg = self.entry.get().strip()

    if not msg or not self.current_chat:

        return

    time_now = datetime.now().strftime("%H:%M")

    try:

        if self.current_chat.startswith("Room:"):

            room = self.current_chat.replace("Room: ", "")

            self.client.sendall(f"GROUP | {room} | {msg}\n".encode())

        else:

            self.client.sendall(f"PRIVATE | {self.current_chat} | {msg}\n".encode())

    except Exception:

        messagebox.showerror("Error", "Lost connection to server.")

        return

    self._add_message(self.current_chat, f"{msg} {time_now}", "me")

    self.entry.delete(0, tk.END)

def send_file(self):

    path = filedialog.askopenfilename()

    if not path or not self.current_chat:

        return

```

```

size = os.path.getsize(path)
name = os.path.basename(path)
if not messagebox.askyesno("Confirm", f"Send {name} ({size // 1024} KB)?"):
    return

target = self.current_chat.replace("Room: ", "")
try:
    header = f"FILE|{target}|{name}|{size}\n"
    with open(path, "rb") as f:
        data = f.read()
    self.client.sendall(header.encode() + data)
except Exception:
    messagebox.showerror("Error", "Lost connection to server.")
    return

chat = self.current_chat
if name.lower().endswith((".png", ".jpg", ".jpeg", ".gif", ".bmp")):
    image = Image.open(path)
    image = image.resize((180, 180), Image.LANCZOS)
    photo = ImageTk.PhotoImage(image)
    self._add_message(chat, photo, "me", True)
else:
    file_msg = {'file': True, 'name': name, 'data': data, 'text': f"File: {name}"}
    self._add_message(chat, file_msg, "me")

def _save_file(self, name, data):
    path = filedialog.asksaveasfilename(defaultextension="", initialfile=name)
    if path:
        with open(path, "wb") as f:
            f.write(data)

```

```

def _save_chat_image(self, photo):
    path = filedialog.asksaveasfilename(defaultextension=".png",
                                         filetypes=[("PNG Image", "*.png")])

    if path:
        img = ImageTk.getimage(photo)
        img.save(path)

# =====
#          RECEIVE LOOP (Buffered)
# =====

def receive_loop(self):
    reader = BufferedReader(self.client)

    while True:
        try:
            header = reader.read_line()

            if not header:
                break

            parts = header.split('|')
            mtype = parts[0]

            # —— USER LIST ——

            if mtype == "LIST":
                users = parts[1].split(',') if len(parts) > 1 else []

                for u in users:
                    if u and u != self.username:
                        self._add_chat_item(u, is_room=False)

            # Remove items for users no longer online
            current_users = set(u for u in users if u and u != self.username)

            for name in list(self.chat_items.keys()):
                if not name.startswith("Room:") and name not in current_users:

```

```

        self.chat_items[name]['frame'].destroy()

        del self.chat_items[name]

# —— ROOM LIST ——
elif mtype == "ROOMS":
    room_list = parts[1].split(',') if len(parts) > 1 else []
    for r in room_list:
        if r:
            self._add_chat_item(f"Room: {r}", is_room=True)

# —— FILE ——
elif mtype == "FILE":
    sender = parts[1]
    fname = parts[2]
    size = int(parts[3])
    data = reader.read_bytes(size)

    chat = sender

    if fname.lower().endswith((".png", ".jpg", ".jpeg", ".gif", ".bmp")):
        image = Image.open(io.BytesIO(data))
        image = image.resize((180, 180), Image.LANCZOS)
        photo = ImageTk.PhotoImage(image)
        self._add_message(chat, photo, "other", True)
    else:
        file_msg = {'file': True, 'name': fname, 'data': data,
                    'text': f"File: {fname}"}
        self._add_message(chat, file_msg, "other")

# —— VIDEO / VOICE DATA ——
elif mtype in ("VIDEO", "VOICE"):
    sender = parts[1]

```

```

size = int(parts[3])

data = reader.read_bytes(size)

self.media_queue.put((mtype, data))


# —— INCOMING CALL ——

elif mtype == "VIDCALL_REQ":

    from_user = parts[1]

    call_type = parts[2] if len(parts) > 2 else 'video'


    # Ring notification

    self._play_ringtone()


    # Show incoming call dialog on main thread

    self.root.after(0, lambda u=from_user, ct=call_type: self._show_incoming_call(u, ct))


# —— CALL ACCEPTED ——

elif mtype == "VIDCALL_ACC":

    from_user = parts[1]

    call_type = parts[2] if len(parts) > 2 else 'video'

    self.root.after(0, lambda u=from_user, ct=call_type: self._on_call_accepted(u, ct))


# —— CALL REJECTED ——

elif mtype == "VIDCALL_REJ":

    from_user = parts[1]

    self.root.after(0, lambda: self._on_call_rejected())


# —— CALL ENDED ——

elif mtype == "VIDCALL_END":

    from_user = parts[1] if len(parts) > 1 else ""

    self.root.after(0, lambda: self._on_call_ended_by_partner())

```

```

# —— TEXT MESSAGES ——

else:

    time_now = datetime.now().strftime("%H:%M")

    if header.startswith("[SYSTEM]"):

        chat = self.current_chat if self.current_chat else "System"

        self._add_message(chat, f"{header} {time_now}", "system")

    elif "[PRIVATE]" in header:

        sender = header.split()[1].replace(":", "")

        msg_text = header.split(":", 1)[1].strip() if ":" in header else header

        self._add_message(sender, f"{msg_text} {time_now}", "other")

    else:

        # Group message

        bracket_end = header.find(']')

        if bracket_end > 0:

            room = header[1:bracket_end]

            chat = f"Room: {room}"

            self._add_message(chat, f"{header} {time_now}", "other")

        else:

            self._add_message(self.current_chat or "System", f"{header} {time_now}", "other")

except ConnectionError:

    self.root.after(0, lambda: messagebox.showerror("Disconnected", "Lost connection to
server."))

    break

except Exception as e:

    print(f"[Receive Error] {e}")

    continue

# =====

#         MEDIA HANDLER

# =====

```

```

def media_handler(self):
    """Processes received VIDEO/VOICE frames from the queue."""
    p = None
    stream = None

    while True:
        try:
            item = self.media_queue.get()

            if item is None:
                break

            mtype, data = item

            if mtype == "VIDEO":
                try:
                    frame = pickle.loads(data)
                    img = cv2.imdecode(frame, cv2.IMREAD_COLOR)
                    if img is not None:
                        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                        pil_img = Image.fromarray(img_rgb)
                        pil_img = pil_img.resize((400, 300), Image.LANCZOS)
                        self.remote_photo = pil_img
                except Exception:
                    pass

            elif mtype == "VOICE":
                try:
                    if HAS_PYAUDIO:
                        if p is None:
                            p = pyaudio.PyAudio()
                            stream = p.open(format=pyaudio.paInt16, channels=1,

```

```

        rate=44100, output=True,
        frames_per_buffer=1024)

    if stream and stream.is_active():
        stream.write(data)
    except Exception:
        pass

except Exception:
    continue

# =====
#          CALL FEATURES
# =====

def video_call(self):
    if not self.current_chat or self.call_active:
        if self.call_active:
            messagebox.showinfo("Call Active", "Already in a call. End current call first.")
        return
    target = self.current_chat.replace("Room: ", "")
    try:
        self.client.sendall(f"VIDCALL_REQ|{target}|video\n".encode())
    except Exception:
        messagebox.showerror("Error", "Lost connection to server.")
        return
    self._add_message(target, "Video call ringing...", "system")

def voice_call(self):
    if not self.current_chat or self.call_active:
        if self.call_active:
            messagebox.showinfo("Call Active", "Already in a call. End current call first.")
        return

```

```

target = self.current_chat.replace("Room: ", "")

try:
    self.client.sendall(f"VIDCALL_REQ|{target}|voice\n".encode())
except Exception:
    messagebox.showerror("Error", "Lost connection to server.")

    return

self._add_message(target, "Voice call ringing...", "system")


def _show_incoming_call(self, from_user, call_type):
    """Show incoming call notification (runs on main thread)."""

    # Create incoming call window

    self.incoming_window = tk.Toplevel(self.root)
    self.incoming_window.title("Incoming Call")
    self.incoming_window.geometry("380x280")
    self.incoming_window.configure(bg=C.CALL_BG)
    self.incoming_window.resizable(False, False)
    self.incoming_window.grab_set()

    # Center on screen

    self.incoming_window.update_idletasks()
    x = self.root.winfo_x() + (self.root.winfo_width() // 2) - 190
    y = self.root.winfo_y() + (self.root.winfo_height() // 2) - 140
    self.incoming_window.geometry(f"{x}x{y}")

    # Avatar

    avatar = tk.Canvas(self.incoming_window, width=80, height=80, bg=C.CALL_BG,
highlightthickness=0)

    avatar.pack(pady=(30, 10))

    avatar.create_oval(5, 5, 75, 75, fill=C.ACCENT, outline=C.ACCENT)

    avatar.create_text(40, 40, text=from_user[:2].upper(), font=("Segoe UI", 20, "bold"),
fill=C.WHITE)

```

Name

```
tk.Label(self.incoming_window, text=from_user, font=("Segoe UI", 16, "bold"),
        bg=C.CALL_BG, fg=C.TEXT_PRIMARY).pack()
```

Call type

```
call_emoji = "📺 Video Call" if call_type == 'video' else "☎ Voice Call"
```

```
tk.Label(self.incoming_window, text=call_emoji, font=("Segoe UI", 12),
        bg=C.CALL_BG, fg=C.TEXT_SECONDARY).pack(pady=5)
```

Buttons

```
btn_frame = tk.Frame(self.incoming_window, bg=C.CALL_BG)
btn_frame.pack(pady=20)
```

```
accept_btn = tk.Button(btn_frame, text="✓ Accept", font=("Segoe UI", 12, "bold"),
                      bg=C.CALL_GREEN, fg=C.WHITE, relief=tk.FLAT, cursor="hand2",
                      width=10, height=1,
                      command=lambda: self._accept_incoming_call(from_user, call_type))
accept_btn.pack(side=tk.LEFT, padx=15)
```

```
reject_btn = tk.Button(btn_frame, text="✗ Reject", font=("Segoe UI", 12, "bold"),
                      bg=C.CALL_RED, fg=C.WHITE, relief=tk.FLAT, cursor="hand2",
                      width=10, height=1,
                      command=lambda: self._reject_incoming_call(from_user, call_type))
reject_btn.pack(side=tk.LEFT, padx=15)
```

Play ringtone

```
self._play_ringtone()
```

```
def _accept_incoming_call(self, from_user, call_type):
```

```
    """Accept an incoming call."""
```

```
    if hasattr(self, 'incoming_window') and self.incoming_window:
```

```

        self.incoming_window.destroy()

        self.incoming_window = None

    try:
        self.client.sendall(f"VIDCALL_ACC|{from_user}|{call_type}\n".encode())
    except Exception:
        messagebox.showerror("Error", "Lost connection to server.")
        return

    self._add_message(from_user, f"Accepted {call_type} call.", "system")
    self._start_call(from_user, call_type)

def _reject_incoming_call(self, from_user, call_type):
    """Reject an incoming call."""
    if hasattr(self, 'incoming_window') and self.incoming_window:
        self.incoming_window.destroy()
        self.incoming_window = None

    try:
        self.client.sendall(f"VIDCALL_REJ|{from_user}|{call_type}\n".encode())
    except Exception:
        pass

    self._add_message(from_user, f"Rejected {call_type} call.", "system")

def _on_call_accepted(self, partner, call_type):
    """Called when the partner accepts our outgoing call."""
    self._add_message(partner, "Call accepted! Connecting...", "system")
    self._start_call(partner, call_type)

def _on_call_rejected(self):

```

```

        """Called when the partner rejects our outgoing call."""
        messagebox.showinfo("Call Rejected", "Your call was rejected.")
        self._add_message(self.call_partner or "Unknown", "Call rejected.", "system")

def _on_call_ended_by_partner(self):
    """Called when the partner ends the call."""
    self._end_call(notify_partner=False)

# ————— START CALL —————
def _start_call(self, partner, call_type):
    """Initialize call state and open call window."""
    self.call_active = True
    self.call_partner = partner
    self.call_type = call_type
    self.call_start_time = time.time()
    self._stop_call.clear()

    self._open_call_window(call_type)

    # Start sending media
    if call_type == 'video':
        threading.Thread(target=self._send_video_and_voice, args=(partner,), daemon=True).start()
    else:
        threading.Thread(target=self._send_voice_only, args=(partner,), daemon=True).start()

# ————— CALL WINDOW —————
def _open_call_window(self, call_type):
    """Open the call window with video/voice UI."""
    self.call_window = tk.Toplevel(self.root)
    self.call_window.title(f"{'Video' if call_type == 'video' else 'Voice'} Call — {self.call_partner}")
    self.call_window.geometry("700x550")

```

```

self.call_window.configure(bg=C.CALL_BG)

self.call_window.protocol("WM_DELETE_WINDOW", self.end_call)


# Center on screen

self.call_window.update_idletasks()

x = self.root.winfo_x() + (self.root.winfo_width() // 2) - 350
y = self.root.winfo_y() + (self.root.winfo_height() // 2) - 275
self.call_window.geometry(f"{x}+{y}")


if call_type == 'video':
    self._build_video_call_ui()
else:
    self._build_voice_call_ui()


# Start timer update

self._update_call_timer()


def _build_video_call_ui(self):
    """Build the video call window UI."""

    # Remote video (large)

    remote_frame = tk.Frame(self.call_window, bg=C.CALL_BG)
    remote_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=(10, 5))

    self.remote_label_widget = tk.Label(remote_frame, bg="#000000",
                                         text="Waiting for video...",
                                         fg=C.TEXT_SECONDARY, font=("Segoe UI", 12))
    self.remote_label_widget.pack(fill=tk.BOTH, expand=True)


    # Local video (small, picture-in-picture)

    pip_frame = tk.Frame(self.call_window, bg=C.CALL_BG)
    pip_frame.place(x=20, rely=0.65, width=200, height=150)

```

```

# Border for local video

border = tk.Frame(pip_frame, bg=C.ACCENT, padx=2, pady=2)
border.pack(fill=tk.BOTH, expand=True)

self.local_label_widget = tk.Label(border, bg="#1a1a2e",
                                    text="Camera",
                                    fg=C.TEXT_SECONDARY, font=("Segoe UI", 9))
self.local_label_widget.pack(fill=tk.BOTH, expand=True)

# Partner name overlay

name_label = tk.Label(self.call_window, text=self.call_partner,
                      font=("Segoe UI", 18, "bold"),
                      bg=C.CALL_BG, fg=C.TEXT_PRIMARY)
name_label.place(x=250, rely=0.02)

# Call timer

self.call_timer_label = tk.Label(self.call_window, text="00:00",
                                  font=("Segoe UI", 12),
                                  bg=C.CALL_BG, fg=C.TEXT_SECONDARY)
self.call_timer_label.place(x=310, rely=0.07)

# Controls

self._build_call_controls(rely=0.92)

# Start video update loop

self._update_local_video()
self._update_remote_video()

def _build_voice_call_ui(self):
    """Build the voice call window UI."""

```

```

# Avatar

avatar_canvas = tk.Canvas(self.call_window, width=120, height=120,
                           bg=C.CALL_BG, highlightthickness=0)
avatar_canvas.pack(pady=(60, 15))
avatar_canvas.create_oval(10, 10, 110, 110, fill=C.ACCENT, outline=C.ACCENT)
avatar_canvas.create_text(60, 60, text=self.call_partner[:2].upper(),
                          font=("Segoe UI", 30, "bold"), fill=C.WHITE)

# Name

tk.Label(self.call_window, text=self.call_partner,
         font=("Segoe UI", 22, "bold"),
         bg=C.CALL_BG, fg=C.TEXT_PRIMARY).pack()

# Call status / timer

self.call_timer_label = tk.Label(self.call_window, text="Calling...",
                                  font=("Segoe UI", 14),
                                  bg=C.CALL_BG, fg=C.TEXT_SECONDARY)
self.call_timer_label.pack(pady=10)

# Animated dots (simulate ringing)

self._animate_call_status()

# Controls

self._build_call_controls(rely=0.82)

def _build_call_controls(self, rely=0.85):
    """Build call control buttons (mute, end call)."""
    ctrl_frame = tk.Frame(self.call_window, bg=C.CALL_BG)
    ctrl_frame.place(relx=0.5, rely=rely, anchor=tk.CENTER)

# Mute button

```

```

self.mute_btn = tk.Button(ctrl_frame, text="🔇", font=("Segoe UI", 18),
                           bg=C.BG_HOVER, fg=C.TEXT_PRIMARY, relief=tk.FLAT,
                           width=4, height=1, cursor="hand2",
                           command=self._toggle_mute,
                           activebackground=C.BORDER)
self.mute_btn.pack(side=tk.LEFT, padx=20)

```

End call button

```

end_btn = tk.Button(ctrl_frame, text="📞 End", font=("Segoe UI", 14, "bold"),
                    bg=C.CALL_RED, fg=C.WHITE, relief=tk.FLAT,
                    width=8, height=1, cursor="hand2",
                    command=self.end_call,
                    activebackground="#c00030")
end_btn.pack(side=tk.LEFT, padx=20)

```

```

def _toggle_mute(self):
    """Toggle microphone mute."""
    self.mic_muted = not self.mic_muted
    if self.mic_muted:
        self.mute_btn.config(text="📞", bg=C.CALL_RED)
    else:
        self.mute_btn.config(text="🔇", bg=C.BG_HOVER)

```

```

def _update_call_timer(self):
    """Update the call duration timer."""
    if not self.call_active or not self.call_window:
        return
    try:
        if self.call_window.winfo_exists():
            elapsed = int(time.time() - self.call_start_time)
            mins = elapsed // 60

```

```

        secs = elapsed % 60

        self.call_timer_label.config(text=f"{mins:02d}:{secs:02d}")

        self.call_window.after(1000, self._update_call_timer)
except Exception:
    pass

```

```

def _animate_call_status(self):
    """Animate the 'Calling...' text for voice calls."""
    if not self.call_active or not self.call_window:
        return

    try:
        if self.call_window.winfo_exists() and self.call_type == 'voice':
            elapsed = int(time.time() - self.call_start_time)
            dots = "." * ((elapsed % 3) + 1)

            self.call_timer_label.config(text=f"Connected {dots}")

            self.call_window.after(1000, self._animate_call_status)
    except Exception:
        pass

```

————— VIDEO UPDATE LOOPS —————

```

def _update_local_video(self):
    """Update local video preview in the call window (runs on main thread)."""
    if not self.call_active or not self.call_window:
        return

    try:
        if self.call_window.winfo_exists() and self.local_cap and self.local_cap.isOpened():
            ret, frame = self.local_cap.read()

            if ret:
                img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
                pil_img = Image.fromarray(img_rgb)
                pil_img = pil_img.resize((196, 146), Image.LANCZOS)

```

```

        self.local_photo = ImageTk.PhotoImage(pil_img)

        if self.local_label_widget and self.local_label_widget.winfo_exists():
            self.local_label_widget.config(image=self.local_photo, text="")
except Exception:
    pass

try:
    if self.call_window.winfo_exists():
        self.call_window.after(50, self._update_local_video)
except Exception:
    pass

def _update_remote_video(self):
    """Update remote video display in the call window (runs on main thread)."""
    if not self.call_active or not self.call_window:
        return

    try:
        if self.call_window.winfo_exists() and self.remote_photo is not None:
            photo = ImageTk.PhotoImage(self.remote_photo)

            if self.remote_label_widget and self.remote_label_widget.winfo_exists():
                self.remote_label_widget.config(image=photo, text="")

                self.remote_label_widget.image = photo # Keep reference

                self.remote_photo = None # Consumed
    except Exception:
        pass

    try:
        if self.call_window.winfo_exists():
            self.call_window.after(50, self._update_remote_video)
    except Exception:
        pass

# ————— SEND VIDEO + VOICE —————

```

```

def _send_video_and_voice(self, target):
    """Capture camera + mic and send to partner (video call)."""
    self.local_cap = cv2.VideoCapture(0)
    if not self.local_cap.isOpened():
        self.root.after(0, lambda: messagebox.showwarning("Camera",
            "Cannot open camera. Video call will continue without local video.))

    p = None
    stream = None
    if HAS_PYAUDIO:
        try:
            p = pyaudio.PyAudio()
            stream = p.open(format=pyaudio.paInt16, channels=1, rate=44100,
                input=True, frames_per_buffer=1024)
        except Exception as e:
            print(f"[Audio Init Error] {e}")
            p = None
            stream = None

    try:
        while self.call_active and not self._stop_call.is_set():
            try:
                # — Video frame —
                if self.local_cap and self.local_cap.isOpened():
                    ret, frame = self.local_cap.read()
                    if ret:
                        # Resize for bandwidth
                        frame = cv2.resize(frame, (320, 240))
                        _, buffer = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 60])
                        data = pickle.dumps(buffer)
                        header = f"VIDEO|{target}|x|{len(data)}\n"

```

```

        self.client.sendall(header.encode() + data)

# — Voice chunk —
if stream:
    if not self.mic_muted:
        audio_data = stream.read(1024, exception_on_overflow=False)
        header = f"VOICE|{target}|x|{len(audio_data)}\n"
        self.client.sendall(header.encode() + audio_data)
    else:
        # Send silence to keep sync
        silence = b'\x00' * 2048
        header = f"VOICE|{target}|x|{len(silence)}\n"
        self.client.sendall(header.encode() + silence)

time.sleep(0.03) # ~30 FPS

except (ConnectionResetError, BrokenPipeError, OSError):
    self.root.after(0, lambda: messagebox.showerror("Call Error", "Connection lost during
call."))

    break

except Exception as e:
    print(f"[Video Send Error] {e}")
    time.sleep(0.05)

finally:
    if self.local_cap:
        self.local_cap.release()

        self.local_cap = None

    try:
        if stream:
            stream.stop_stream()

```

```

        stream.close()

    if p:
        p.terminate()

except Exception:
    pass

# ————— SEND VOICE ONLY —————

def _send_voice_only(self, target):
    """Capture mic and send audio only (voice call)."""
    if not HAS_PYAUDIO:
        self.root.after(0, lambda: messagebox.showwarning("Audio",
            "PyAudio not installed. Voice call requires PyAudio.\nInstall with: pip install pyaudio"))
        return

    p = None
    stream = None

    try:
        p = pyaudio.PyAudio()
        stream = p.open(format=pyaudio.paInt16, channels=1, rate=44100,
            input=True, frames_per_buffer=1024)
    except Exception as e:
        print(f"[Audio Init Error] {e}")
        self.root.after(0, lambda: messagebox.showwarning("Audio", f"Cannot open microphone: {e}"))
        return

    try:
        while self.call_active and not self._stop_call.is_set():
            try:
                if not self.mic_muted:
                    audio_data = stream.read(1024, exception_on_overflow=False)

```

```

        header = f"VOICE | {target} | x | {len(audio_data)}\n"
        self.client.sendall(header.encode() + audio_data)
    else:
        silence = b'\x00' * 2048
        header = f"VOICE | {target} | x | {len(silence)}\n"
        self.client.sendall(header.encode() + silence)

    time.sleep(0.02)

except (ConnectionResetError, BrokenPipeError, OSError):
    self.root.after(0, lambda: messagebox.showerror("Call Error", "Connection lost during
call.))
    break
except Exception as e:
    print(f"[Voice Send Error] {e}")
    time.sleep(0.05)

finally:
    try:
        stream.stop_stream()
        stream.close()
        p.terminate()
    except Exception:
        pass

# ————— END CALL —————

def end_call(self):
    """End the current call (user initiated)."""
    self._end_call(notify_partner=True)

def _end_call(self, notify_partner=True):

```

```
"""Internal: end call and clean up."""
if not self.call_active and not self.call_partner:
    return

self.call_active = False
self._stop_call.set()

# Notify partner
if notify_partner and self.call_partner:
    try:
        self.client.sendall(f"VIDCALL_END|{self.call_partner}\n".encode())
    except Exception:
        pass

# Release camera
if self.local_cap:
    self.local_cap.release()
    self.local_cap = None

# Close call window
if self.call_window:
    try:
        self.call_window.destroy()
    except Exception:
        pass
    self.call_window = None

# Reset remote photo
self.remote_photo = None

# Add system message
```

```
partner = self.call_partner or "Unknown"
self._add_message(partner, "Call ended.", "system")
```

```
# Reset state
```

```
self.call_partner = None
```

```
self.call_type = None
```

```
self.call_start_time = None
```

```
self.mic_muted = False
```

```
# ————— RINGTONE —————
```

```
def _play_ringtone(self):
```

```
    """Play a ringtone sound (cross-platform)."""
```

```
    def _ring():
```

```
        try:
```

```
            if not HAS_PYAUDIOIO:
```

```
                return
```

```
            p = pyaudio.PyAudio()
```

```
            stream = p.open(format=pyaudio.paInt16, channels=1, rate=44100,
```

```
                             output=True, frames_per_buffer=1024)
```

```
            import math
```

```
            for _ in range(3):
```

```
                for freq in [600, 800]:
```

```
                    for i in range(0, 4410):
```

```
                        val = int(32767 * 0.3 * math.sin(2 * math.pi * freq * i / 44100))
```

```
                        stream.write(val.to_bytes(2, byteorder='little', signed=True))
```

```
                    time.sleep(0.15)
```

```
            stream.stop_stream()
```

```
            stream.close()
```

```
            p.terminate()
```

```
        except Exception:
```

```
            pass
```

```

threading.Thread(target=_ring, daemon=True).start()

# =====
#          ROOM MANAGEMENT
# =====

def _room_menu(self):
    """Show room management menu."""
    menu = tk.Menu(self.root, tearoff=0, bg=C.BG_SIDEBAR, fg=C.TEXT_PRIMARY,
                    activebackground=C.ACCENT, activeforeground=C.WHITE,
                    font=("Segoe UI", 11))

    menu.add_command(label="🏠 Create Room", command=self._create_room)
    menu.add_command(label="📖 Join Room", command=self._join_room)
    menu.add_command(label="👋 Leave Room", command=self._leave_room)

    # Show at button position
    menu.tk_popup(self.root.winfo_pointerx(), self.root.winfo_pointery())

def _create_room(self):
    room = simpledialog.askstring("Create Room", "Enter room name:", parent=self.root)
    if room:
        try:
            self.client.sendall(f"CREATE|{room}\n".encode())
        except Exception:
            messagebox.showerror("Error", "Lost connection to server.")
            return
        self._add_chat_item(f"Room: {room}", is_room=True)

def _join_room(self):
    # Show available rooms

```

```

room_names = [name.replace("Room: ", "") for name in self.chat_items.keys()
                if name.startswith("Room:")]
if not room_names:
    messagebox.showinfo("No Rooms", "No rooms available to join.")
    return

room = simpledialog.askstring("Join Room",
                              f"Available rooms:\n{' '.join(room_names)}\n\nEnter room name:",
                              parent=self.root)

if room:
    try:
        self.client.sendall(f"JOIN|{room}\n".encode())
    except Exception:
        messagebox.showerror("Error", "Lost connection to server.")

def _leave_room(self):
    if not self.current_chat or not self.current_chat.startswith("Room:"):
        messagebox.showinfo("Info", "Select a room to leave.")
        return

    room = self.current_chat.replace("Room: ", "")
    try:
        self.client.sendall(f"LEAVE|{room}\n".encode())
    except Exception:
        messagebox.showerror("Error", "Lost connection to server.")
        return

# Remove from UI
if self.current_chat in self.chat_items:
    self.chat_items[self.current_chat]['frame'].destroy()
    del self.chat_items[self.current_chat]

self._show_welcome()

```

```
self.current_chat = None

self.header_name.config(text="Select a chat")

self.header_status.config(text="")
```

```
# ----- MAIN -----
```

```
def main():

    root = tk.Tk()

    try:

        root.state('zoomed') # Maximize on Windows

    except Exception:

        try:

            root.attributes('-zoomed', True) # Maximize on Linux

        except Exception:

            pass

    app = ChatClient(root)

    root.mainloop()
```

```
if __name__ == "__main__":

    main()
```